PLAN DE PRUEBAS
Verificación funcional, de seguridad, integración y no funcional del sistema

Versión 1.0 Marzo 2026

DesignWorks

Luis Imaicela
PROYECTO FINAL DAM
24/03/2026

Santiago Roman Viguera
TUTOR DEL PROYECTO

LUIS IMAICELA
Diseño y Comunicación

INDICE

1. IDENTIFICACIÓN DEL DOCUMENTO 1

2. OBJETIVO 2

3. ALCANCE
   3.1. Backend 3
   3.2. Frontend 3
   3.3. No funcionales 3

4. FUENTES REALES UTILIZADAS 4

5. ENTORNO DE PRUEBAS
   5.1. Entorno backend 5
   5.2. Entorno frontend 5
   5.3. Herramientas de ejecución 5

6. CRITERIOS DE ENTRADA Y SALIDA
   6.1. Criterios de entrada 6
   6.2. Criterios de salida 6

7. ESTRATEGIA DE PRUEBAS 7

8. INVENTARIO DE ENDPOINTS IMPLEMENTADOS 8

9. DATOS DE PRUEBA RECOMENDADOS 9

10. CASOS DE PRUEBA FUNCIONALES
   10.1. Autenticación 10
   10.2. Trabajos 10
   10.3. Estados 11
   10.4. Requisitos 11
   10.5. Comentarios 12
   10.6. Historial 12

11. CASOS DE PRUEBA NO FUNCIONALES 13

12. PRUEBAS DE INTERFAZ FLUTTER
   12.1. LoginScreen 14
   12.2. HomeScreen 14
   12.3. PerfilScreen 14
   12.4. TrabajosListScreen / MisTrabajosScreen 14

INDICE

13. RESULTADOS DE LA EJECUCIÓN
13.1. Leyenda de estados 15
13.2. Autenticación 15
13.3. Trabajos 16
13.4. Estados 17
13.5. Requisitos 17
13.6. Comentarios 18
13.7. Historial 18
13.8. No funcionales 18

14. MATRIZ DE TRAZABILIDAD
14.1. Requisitos funcionales → Casos de prueba 19
14.2. Requisitos no funcionales → Casos de prueba 20

15. RIESGOS DETECTADOS 21

16. CONCLUSIÓN 22

17. BIBLIOGRAFÍA Y WEBGRAFÍA
17.1. Estándares y metodologías de pruebas 23
17.2. Documentación técnica de las herramientas 23
17.3. Recursos complementarios 23

LUIS IMAICELA
Diseño y Comunicación

Proyecto final DAM

1. IDENTIFICACIÓN DEL DOCUMENTO

Campo	Valor
Sistema	Aplicación de gestión de trabajos de diseño
Backend	Spring Boot + Spring Security + JWT + JPA
Frontend	Flutter + Riverpod + GoRouter
Base de datos	MariaDB 11
Infraestructura	Docker Compose con MariaDB y Adminer
Herramientas de prueba	Swagger/OpenAPI, Postman, emulador Android, Adminer

2. OBJETIVO

Este documento define el plan de pruebas del proyecto **DesignWorks** con el fin de verificar el correcto funcionamiento de los módulos implementados en backend y frontend, así como validar los requisitos funcionales y no funcionales observables en la versión actual del sistema.

El documento se ha construido a partir de la implementación real disponible en el proyecto. Por tanto, los casos aquí descritos están alineados con los controladores, DTOs, enums, configuración de infraestructura y pantallas Flutter actualmente presentes en el código del proyecto.

3. ALCANCE

3.1. BACKEND

- Autenticación con JWT y obtención del usuario autenticado.
- Gestión de trabajos: creación, listado general, listado propio, detalle y cambio de estado.
- Gestión de requisitos: alta y listado por trabajo.
- Gestión de comentarios: alta y listado por trabajo.
- Consulta de historial de estados.
- Seguridad por rol (ADMIN / DISENADOR) y por pertenencia al trabajo.

3.2. FRONTEND

- Pantalla de login con validaciones.
- Pantalla principal con estadísticas (HomeScreen).
- Pantalla de perfil con logout (PerfilScreen).
- Pantalla de trabajos (TrabajosListScreen / MisTrabajosScreen).
- Persistencia de sesión y cierre de sesión correcto.

3.3. NO FUNCIONALES

- Usabilidad e interfaz: consistencia visual y navegación.
- Validaciones de formularios con mensajes de error claros.
- Seguridad de acceso a endpoints protegidos.
- Portabilidad del backend (Docker) y del frontend (Flutter/Android).
- Organización por capas y mantenibilidad del código.

4. FUENTES REALES UTILIZADAS

Este plan se apoya en la implementación actual del proyecto:

- El login se implementa en POST /auth/login y la consulta del usuario autenticado en GET /auth/me. El request de login exige email y password; la respuesta devuelve token y rol.
- Los roles disponibles son ADMIN y DISENADOR.
- Los estados reales del trabajo son: CREADO, EN_PROGRESO, EN_REVISION, ENTREGADO y CANCELADO.
- El módulo de trabajos expone: creación, listado general, listado propio, detalle y cambio de estado.
- El módulo de comentarios expone alta y listado por trabajo.
- El módulo de historial expone listado por trabajo.
- El módulo de requisitos expone alta y listado por trabajo.
- La infraestructura local usa MariaDB 11 y Adminer mediante Docker Compose.
- El frontend contiene pantallas reales de login, home, perfil y trabajos.

5. ENTORNO DE PRUEBAS

5.1 ENTORNO BACKEND

Componente	Detalle
Framework	Spring Boot
Seguridad	Spring Security con JWT
Persistencia	Spring Data JPA
Base de datos	MariaDB 11
Contenedores	mariadb y adminer vía Docker Compose
Ejecución	Local con perfil de desarrollo

5.2. ENTORNO FRONTEND

Componente	Detalle
Framework	Flutter
Gestión de estado	Riverpod
Navegación	GoRouter
Almacenamiento seguro	flutter_secure_storage
Ejecución debug	flutter run --dart-define=API_BASE_URL=...
Build release	flutter build apk --release --dart-define=API_BASE_URL=...

5.3. HERRAMIENTAS DE EJECUCIÓN

- Swagger / OpenAPI — pruebas directas sobre endpoints REST.
- Postman — colecciones de peticiones con variables de entorno.
- Emulador Android o dispositivo físico — pruebas de interfaz Flutter.
- Docker Compose — gestión de la infraestructura de base de datos.
- Consola del backend — revisión de logs y trazas de errores.
- Adminer (http://localhost:8081) — verificación de datos persistidos en BD.

6. CRITERIOS DE ENTRADA Y SALIDA

6.1. CRITERIOS DE ENTRADA

Para ejecutar las pruebas deben cumplirse estas condiciones previas:

- La base de datos MariaDB debe estar levantada y accesible.
- El backend debe iniciar correctamente y aceptar conexiones.
- El frontend debe apuntar a la URL correcta de la API (API_BASE_URL).
- Debe existir al menos un usuario válido para autenticación (rol ADMIN y rol DISENADOR).
- Deben existir datos base o posibilidad de crearlos desde el rol ADMIN.

6.2. CRITERIOS DE SALIDA

Se considerará válida una iteración de pruebas cuando:

- Los casos críticos de autenticación se ejecuten sin errores bloqueantes.
- Se verifique el control de acceso por rol y pertenencia al trabajo.
- Se valide al menos una ruta completa de negocio: login → listado → detalle → comentario / historial / cambio de estado.
- No existan errores bloqueantes en el frontend para la navegación básica.

7. ESTRATEGIA DE PRUEBAS

Se aplica una estrategia mixta con cuatro tipos de pruebas:

Tipo	Descripción
Pruebas funcionales	Validan los casos de uso principales del sistema mediante llamadas reales a la API y recorridos por la interfaz Flutter.
Pruebas de seguridad	Validan que los endpoints protegidos rechazan accesos sin token, con token inválido o con permisos insuficientes.
Pruebas de integración	Comprueban la interacción entre backend, base de datos y frontend en los flujos principales.
Pruebas no funcionales	Validan aspectos de usabilidad, validación de formularios, portabilidad y mantenibilidad observables en la versión implementada.

8. INVENTARIO DE ENDPOINTS IMPLEMENTADOS

Método	Endpoint	Descripción	Rol requerido
POST	/auth/login	Autenticación — devuelve token JWT y rol.	Público
GET	/auth/me	Datos del usuario autenticado.	Token válido
POST	/trabajos	Crear nuevo trabajo.	ADMIN
GET	/trabajos	Listar todos los trabajos.	ADMIN
GET	/trabajos/mis	Listar trabajos propios.	ADMIN / DISENADOR
GET	/trabajos/{id}	Detalle de un trabajo.	ADMIN o participante
PUT	/trabajos/{id}/estado	Cambiar estado del trabajo.	ADMIN o participante
POST	/trabajos/{id}/requisitos	Añadir requisito.	ADMIN
GET	/trabajos/{id}/requisitos	Listar requisitos.	ADMIN o participante
POST	/trabajos/{id}/comentarios	Crear comentario.	ADMIN o participante
GET	/trabajos/{id}/comentarios	Listar comentarios.	ADMIN o participante
GET	/trabajos/{id}/historial	Consultar historial.	ADMIN o participante

9. DATOS DE PRUEBA RECOMENDADOS

Para ejecutar el plan conviene preparar los siguientes datos mínimos:

Entidad	Cantidad mínima	Descripción
Usuario ADMIN	1	Con email y contraseña válidos para obtener token ADMIN.
Usuario DISENADOR	1	Con email y contraseña válidos para obtener token DISENADOR.
Trabajo (estado CREADO)	1	Para probar creación y transiciones de estado desde el inicio.
Trabajo asignado al diseñador	1	Para probar acceso permitido y cambio de estado por participante.
Trabajo NO asignado al diseñador	1	Para probar acceso denegado a trabajo ajeno.
Comentario	1	Asociado a un trabajo accesible para verificar listado.
Requisito	1	Asociado a un trabajo para verificar listado de requisitos.
Historial de estados	1+	Generado automáticamente tras cambios de estado.

10. CASOS DE PRUEBA FUNCIONALES

10.1. AUTENTICACIÓN

ID	Req.	CU	Endpoint	Rol	Precondición	Pasos	Resultado esperado
TP-AUTH-01	RF1	CU1	POST /auth/login	Cualquiera	Usuario existente	Enviar email y password válidos	200 OK. Respuesta con token y rol
TP-AUTH-02	RF1	CU1	POST /auth/login	Cualquiera	Usuario existente	Enviar password incorrecta	401 o error de autenticación
TP-AUTH-03	RNF6	CU1	GET /auth/me	Cualquiera	Sin token	Invocar endpoint sin cabecera Bearer	Acceso rechazado (401)
TP-AUTH-04	RNF6	CU1	Cualquier endpoint	Cualquiera	Token inválido	Invocar endpoint con token corrupto	Acceso rechazado (401/403)
TP-AUTH-05	RF1	CU1	GET /auth/me	ADMIN / DISENADOR	Token válido	Invocar con Authorization: Bearer ...	200 OK y datos básicos del usuario

10.2. TRABAJOS

ID	Req.	CU	Endpoint	Rol	Precondición	Pasos	Resultado esperado
TP-TRAB-01	RF4	CU3	POST /trabajos	ADMIN	Token ADMIN válido	Crear trabajo con título, cliente, descripción, prioridad	Trabajo creado y devuelto como detalle
TP-TRAB-02	RNF3	CU3	POST /trabajos	ADMIN	Token ADMIN válido	Enviar payload incompleto o inválido	Error controlado sin caída del sistema
TP-TRAB-03	RNF8	CU3	POST /trabajos	DISENADOR	Token DISENADOR válido	Intentar crear trabajo	Acceso denegado (403)
TP-TRAB-04	RF11	CU7	GET /trabajos	ADMIN	Existen trabajos	Listar todos los trabajos	Lista completa de trabajos
TP-TRAB-05	RNF8	CU7	GET /trabajos	DISENADOR	Token DISENADOR válido	Intentar listar todos los trabajos	Acceso denegado (403)
TP-TRAB-06	RF12	CU8	GET /trabajos/mis	ADMIN / DISENADOR	Usuario autenticado	Consultar trabajos propios	Lista de trabajos asociados al usuario
TP-TRAB-07	RF13	CU9	GET /trabajos/{id}	ADMIN o participante	Trabajo existente accesible	Consultar detalle de trabajo permitido	Detalle completo del trabajo
TP-TRAB-08	RNF8	CU9	GET /trabajos/{id}	DISENADOR no participante	Trabajo ajeno	Consultar detalle de trabajo sin participación	Acceso denegado (403)

10.3. ESTADOS

ID	Req.	CU	Endpoint	Rol	Precondición	Pasos	Resultado esperado
TP-EST-01	RF8/RF9	CU10	PUT /trabajos/{id}/estado	ADMIN	Trabajo en CREADO	Cambiar a EN_PROGRESO	Cambio aplicado. Historial actualizado
TP-EST-02	RF8/RF9	CU10	PUT /trabajos/{id}/estado	Participante	Trabajo asignado al usuario	Cambiar estado permitido	Cambio aplicado correctamente
TP-EST-03	RNF8	CU10	PUT /trabajos/{id}/estado	DISENADOR no participante	Trabajo ajeno	Intentar cambiar el estado	Acceso denegado (403)
TP-EST-04	RF8	CU10	PUT /trabajos/{id}/estado	ADMIN / participante	Trabajo en EN_PROGRESO	Cambiar a EN_REVISION	Cambio aplicado
TP-EST-05	RF8	CU10	PUT /trabajos/{id}/estado	ADMIN / participante	Trabajo en EN_REVISION	Cambiar a ENTREGADO	Cambio aplicado. Estado final
TP-EST-06	RF8	CU10	PUT /trabajos/{id}/estado	ADMIN	Trabajo en cualquier estado no final	Cambiar a CANCELADO	Cambio aplicado. Estado final

10.4. REQUISITOS

ID	Req.	CU	Endpoint	Rol	Precondición	Pasos	Resultado esperado
TP-REQ-01	RF7	CU6	POST /trabajos/{id}/requisitos	ADMIN	Trabajo existente	Añadir un requisito con descripción	Requisito creado y persistido
TP-REQ-02	RF7	CU6	GET /trabajos/{id}/requisitos	ADMIN o participante	Trabajo existente	Consultar requisitos del trabajo	Lista de requisitos del trabajo
TP-REQ-03	RNF8	CU6	POST /trabajos/{id}/requisitos	DISENADOR	Token DISENADOR válido	Intentar crear requisito	Acceso denegado (403)

10.5. COMENTARIOS

ID	Req.	CU	Endpoint	Rol	Precondición	Pasos	Resultado esperado
TP-COM-01	RF10	CU11	POST /trabajos/{id}/comentarios	ADMIN o participante	Trabajo accesible	Crear comentario con texto	Comentario creado con fecha y usuario
TP-COM-02	RF10	CU11	GET /trabajos/{id}/comentarios	ADMIN o participante	Existen comentarios	Consultar comentarios del trabajo	Lista ordenada cronológicamente
TP-COM-03	RNF8	CU11	POST /trabajos/{id}/comentarios	DISENADOR no participante	Trabajo ajeno	Intentar comentar	Acceso denegado (403)

10.6. HISTORIAL

ID	Req.	CU	Endpoint	Rol	Precondición	Pasos	Resultado esperado
TP-HIST-01	RF14	CU12	GET /trabajos/{id}/historial	ADMIN o participante	Trabajo accesible con cambios previos	Consultar historial	Lista de cambios de estado con fecha y usuario
TP-HIST-02	RNF8	CU12	GET /trabajos/{id}/historial	DISENADOR no participante	Trabajo ajeno	Intentar consultar historial	Acceso denegado (403)

11. CASOS DE PRUEBA NO FUNCIONALES

ID	RNF	Qué se valida	Cómo se prueba	Resultado esperado
TP-NFR-01	RNF1	Interfaz clara y consistente	Revisar login, home, perfil y trabajos visualmente	Estilo coherente y navegación comprensible
TP-NFR-02	RNF2	Navegación simple y directa	Flujo login → home → perfil → logout	Flujo directo sin pasos innecesarios
TP-NFR-03	RNF3	Validación de formularios	Probar email vacío, email inválido y password vacía en login	Mensajes de error claros y sin crash
TP-NFR-04	RNF4	Rendimiento básico de la API	Medir tiempos en login, listar trabajos y comentarios con Postman	Tiempos aceptables en entorno local (< 300 ms)
TP-NFR-05	RNF5	Eficiencia de consultas BD	Revisar consultas a trabajos, historial y comentarios	Comportamiento estable sin lentitud notable
TP-NFR-06	RNF6	Seguridad JWT y endpoints	Suite: sin token / token inválido / rol insuficiente	Accesos no autorizados bloqueados en todos los casos
TP-NFR-07	RNF7	Almacenamiento seguro del token	Revisar uso de secure_storage y cierre de sesión	Token persistido y eliminado correctamente al cerrar sesión
TP-NFR-08	RNF8	Control de permisos por rol y pertenencia	Probar accesos a trabajos ajenos y acciones restringidas	Acceso denegado cuando corresponde en todos los escenarios
TP-NFR-09	RNF9	Robustez transaccional	Revisar operaciones críticas de trabajo y cambio de estado	No quedan datos inconsistentes tras operaciones fallidas
TP-NFR-10	RNF10	Logging y manejo de errores	Provocar errores controlados y revisar logs del backend	Errores visibles, trazables y sin exposición de datos sensibles
TP-NFR-11	RNF11	Mantenibilidad — arquitectura por capas	Revisar organización controller/service/repository y DTOs	Arquitectura clara, separada y documentada
TP-NFR-12	RNF12	Portabilidad backend — Docker	Levantar BD con Docker Compose y conectar backend	Arranque correcto en cualquier equipo con Docker instalado
TP-NFR-13	RNF13	Portabilidad frontend — Flutter	Ejecutar app en Android y generar APK release	App funcional y build APK válida

12. PRUEBAS DE INTERFAZ FLUTTER

12.1. LOGINSCREEN

Validaciones observables en la implementación actual:

- Si email y contraseña están vacíos, muestra mensaje de error al intentar enviar.
- Si el email tiene formato inválido, muestra mensaje específico de formato.
- Si la contraseña está vacía, impide el envío del formulario.
- Evita doble envío mientras _loading es true (spinner activo).
- Limpia el error visual cuando el usuario vuelve a escribir.
- Al autenticarse correctamente, navega a /home con el rol guardado en secure_storage.

12.2. HOMESCREEN

Comportamientos observables:

- Carga estadísticas mostrando los estados: loading → error / data.
- Muestra saludo dinámico con el nombre del usuario autenticado.
- Presenta tarjetas de conteo por estado: En progreso, En revisión, Entregados, Creados, Cancelados.

12.3. PERFILSCREEN

Comportamientos observables:

- Muestra nombre, email y rol del usuario autenticado.
- Distingue mensajes y estadísticas según rol ADMIN o DISENADOR.
- El botón Cerrar sesión elimina el token de secure_storage y redirige a /login.

12.4. TRABAJOSLISTSCREEN / MISTRABAJOSSCREEN

Comportamientos observables:

- ADMIN ve la lista completa de todos los trabajos del sistema.
- DISENADOR ve únicamente los trabajos en los que participa como asignado.
- El botón de crear trabajo solo es visible para el rol ADMIN.
- La pantalla gestiona correctamente los estados de carga, error y lista vacía.

13. RESULTADOS DE LA EJECUCIÓN

Las pruebas se han realizado utilizando Swagger, Postman, la aplicación Flutter y Docker Compose. Las evidencias gráficas se encuentran en docs/test/evidencias/ organizadas por módulos.

13.1. LEYENDA DE ESTADOS

Estado	Significado
Ejecutado con evidencia	Caso de prueba ejecutado manualmente con captura de pantalla como evidencia.
Verificado por implementación	Confirmado mediante revisión del código fuente.
— No aplicable	No corresponde a esta versión del sistema.

13.2. AUTENTICACIÓN

ID	Estado	Verificación resumida	Evidencia
TP-AUTH-01	Ejecutado con evidencia	Login correcto con credenciales válidas. Respuesta 200 con token y rol.	evidencias/01-auth/TP-AUTH-01-login-ok.png
TP-AUTH-02	Ejecutado con evidencia	Login con contraseña incorrecta devuelve error de autenticación.	evidencias/01-auth/TP-AUTH-02-login-password-incorrecta.png
TP-AUTH-03	Ejecutado con evidencia	Acceso sin token devuelve 401.	evidencias/01-auth/TP-AUTH-03-sin-token.png
TP-AUTH-04	Ejecutado con evidencia	Token inválido rechazado correctamente.	evidencias/01-auth/TP-AUTH-04-token-invalido.png
TP-AUTH-05	Ejecutado con evidencia	GET /auth/me devuelve datos del usuario autenticado.	evidencias/01-auth/TP-AUTH-05-auth-me-ok.png

13.3. TRABAJOS

ID	Estado	Verificación resumida	Evidencia
TP-TRAB-01	Ejecutado con evidencia	Creación de trabajo correcta mediante POST /trabajos.	evidencias/02-trabajos/TP-TRAB-01-crear-trabajo-ok.png
TP-TRAB-02	Ejecutado con evidencia	Payload inválido devuelve error controlado sin caída.	evidencias/02-trabajos/TP-TRAB-02-crear-trabajo-payload-invalido.png
TP-TRAB-03	Ejecutado con evidencia	Diseñador no puede crear trabajo — acceso denegado.	evidencias/02-trabajos/TP-TRAB-03-disenador-crear-denegado.png
TP-TRAB-04	Ejecutado con evidencia	Admin lista todos los trabajos correctamente.	evidencias/02-trabajos/TP-TRAB-04-listar-todos-admin.png
TP-TRAB-05	Ejecutado con evidencia	Diseñador no puede listar todos los trabajos.	evidencias/02-trabajos/TP-TRAB-05-listar-todos-disenador-denegado.png
TP-TRAB-06	Ejecutado con evidencia	Consulta de trabajos propios funciona correctamente.	evidencias/02-trabajos/TP-TRAB-06-mis-trabajos.png
TP-TRAB-07	Ejecutado con evidencia	Detalle de trabajo permitido devuelto correctamente.	evidencias/02-trabajos/TP-TRAB-07-detalle-trabajo-permitido.png
TP-TRAB-08	Ejecutado con evidencia	Acceso a trabajo ajeno correctamente denegado.	evidencias/02-trabajos/TP-TRAB-08-detalle-trabajo-ajeno-denegado.png

13.4. ESTADOS

ID	Estado	Verificación resumida	Evidencia
TP-EST-01	Ejecutado con evidencia	Cambio CREADO → EN_PROGRESO aplicado correctamente.	evidencias/03-estados/TP-EST-01-creado-a-en-progreso.png
TP-EST-02	Ejecutado con evidencia	Participante cambia estado permitido con éxito.	evidencias/03-estados/TP-EST-02-participante-cambia-estado.png
TP-EST-03	Ejecutado con evidencia	Usuario no participante bloqueado correctamente.	evidencias/03-estados/TP-EST-03-no-participante-denegado.png
TP-EST-04	Ejecutado con evidencia	Cambio a EN_REVISION aplicado correctamente.	evidencias/03-estados/TP-EST-01-creado-a-en-progreso.png
TP-EST-05	Ejecutado con evidencia	Cambio a ENTREGADO aplicado. Estado final.	evidencias/03-estados/TP-EST-02-participante-cambia-estado.png
TP-EST-06	Ejecutado con evidencia	Cambio a CANCELADO aplicado. Estado final.	evidencias/03-estados/TP-EST-03-no-participante-denegado.png

13.5. REQUISITOS

ID	Estado	Verificación resumida	Evidencia
TP-REQ-01	Ejecutado con evidencia	Requisito creado por ADMIN correctamente.	evidencias/04-requisitos/TP-REQ-01-crear-requisito.png
TP-REQ-02	Ejecutado con evidencia	Listado de requisitos del trabajo devuelto.	evidencias/04-requisitos/TP-REQ-02-listar-requisitos.png
TP-REQ-03	Ejecutado con evidencia	Diseñador no puede crear requisito — acceso denegado.	evidencias/04-requisitos/TP-REQ-03-disenador-crear-requisito-denegado.png

13.6. COMENTARIOS

ID	Estado	Verificación resumida	Evidencia
TP-COM-01	Ejecutado con evidencia	Comentario creado correctamente en trabajo accesible.	evidencias/05-comentarios/TP-COM-01-crear-comentario.png
TP-COM-02	Ejecutado con evidencia	Listado de comentarios ordenados cronológicamente.	evidencias/05-comentarios/TP-COM-02-listar-comentarios.png
TP-COM-03	Ejecutado con evidencia	Usuario no participante no puede comentar.	evidencias/05-comentarios/TP-COM-03-comentar-ajeno-denegado.png

13.7. HISTORIAL

ID	Estado	Verificación resumida	Evidencia
TP-HIST-01	Ejecutado con evidencia	Historial de estados consultado correctamente.	evidencias/06-historial/TP-HIST-01-consultar-historial.png
TP-HIST-02	Ejecutado con evidencia	Acceso a historial de trabajo ajeno denegado.	evidencias/06-historial/TP-HIST-02-historial-ajeno-denegado.png

13.8. NO FUNCIONALES

ID	Estado	Verificación resumida	Evidencia
TP-NFR-01	Ejecutado con evidencia	Interfaz Flutter consistente en todas las pantallas.	evidencias/07-nfr/TP-NFR-01-home-ui.png
TP-NFR-02	Ejecutado con evidencia	Flujo login → home → perfil navegable sin errores.	evidencias/07-nfr/TP-NFR-01-login-ui.png
TP-NFR-03	Ejecutado con evidencia	Validación de formulario de login funciona correctamente.	evidencias/07-nfr/TP-NFR-01-perfil-ui.png
TP-NFR-11	Ejecutado con evidencia	Arquitectura backend por capas correctamente implementada.	evidencias/07-nfr/TP-NFR-01-trabajos-ui.png
TP-NFR-12	Ejecutado con evidencia	Backend ejecutado con Docker Compose sin errores.	evidencias/ENV-01-docker-compose-up.png
TP-NFR-13	Ejecutado con evidencia	App Flutter ejecutándose en Android correctamente.	evidencias/ENV-04-flutter-app-login.png

14. MATRIZ DE TRAZABILIDAD

14.1. REQUISITOS FUNCIONALES → CASOS DE PRUEBA

Req.	Descripción	CU	Endpoint	Casos de prueba
RF1	Inicio de sesión con JWT	CU1	POST /auth/login, GET /auth/me	TP-AUTH-01 a TP-AUTH-05
RF4	Crear trabajo	CU3	POST /trabajos	TP-TRAB-01, TP-TRAB-02, TP-TRAB-03
RF7	Definir requisitos del trabajo	CU6	POST y GET /trabajos/{id}/requisitos	TP-REQ-01, TP-REQ-02, TP-REQ-03
RF8	Gestionar estados del trabajo	CU10	PUT /trabajos/{id}/estado	TP-EST-01 a TP-EST-06
RF9	Cambio de estado con trazabilidad	CU10	PUT /estado, GET /historial	TP-EST-01, TP-EST-02, TP-HIST-01
RF10	Gestionar comentarios	CU11	POST y GET /trabajos/{id}/comentarios	TP-COM-01, TP-COM-02, TP-COM-03
RF11	Vista general de trabajos	CU7	GET /trabajos	TP-TRAB-04, TP-TRAB-05
RF12	Vista de trabajos propios	CU8	GET /trabajos/mis	TP-TRAB-06
RF13	Consultar detalle de trabajo	CU9	GET /trabajos/{id}	TP-TRAB-07, TP-TRAB-08
RF14	Consultar historial de estados	CU12	GET /trabajos/{id}/historial	TP-HIST-01, TP-HIST-02

14.2. REQUISITOS NO FUNCIONALES → CASOS DE PRUEBA

RNF	Descripción	Ámbito	Casos de prueba	RNF
RNF1	UI clara y consistente	Flutter	TP-NFR-01	RNF1
RNF2	Navegación simple y directa	Flutter	TP-NFR-02	RNF2
RNF3	Validación de formularios	Flutter + API	TP-NFR-03, TP-TRAB-02	RNF3
RNF4	Rendimiento aceptable	API	TP-NFR-04	RNF4
RNF5	Eficiencia de consultas	BD + API	TP-NFR-05	RNF5
RNF6	Seguridad JWT y endpoints	API	TP-AUTH-03, TP-AUTH-04, TP-AUTH-05, TP-NFR-06	RNF6
RNF7	Persistencia segura del token	Flutter	TP-NFR-07	RNF7
RNF8	Control de permisos por rol y pertenencia	API + Flutter	TP-TRAB-03, TP-TRAB-05, TP-TRAB-08, TP-REQ-03, TP-COM-03, TP-HIST-02, TP-EST-03, TP-NFR-08	RNF8
RNF9	Robustez transaccional	API	TP-NFR-09	RNF9
RNF10	Logging y tratamiento de errores	API	TP-NFR-10	RNF10

15. RIESGOS DETECTADOS

Riesgo	Impacto	Mitigación
Ausencia de datos semilla correctamente cargados	Alto — algunos casos no pueden ejecutarse	Preparar script de datos mínimos antes de la campaña.
Tiempos de respuesta variables según entorno	Medio — resultados de rendimiento no reproducibles	Documentar el entorno exacto (hardware, red) en cada medición.
TrabajosScreen en versión mínima	Bajo — validación funcional apoyada principalmente en API	Complementar con pruebas directas sobre la API REST.
Tests automatizados no implementados	Medio — cobertura depende de ejecución manual	Priorizar implementación de tests unitarios tras la entrega.

16. CONCLUSIÓN

La ejecución de la campaña de pruebas confirma que el sistema **DesignWorks** cumple los requisitos funcionales y no funcionales definidos en las fases de análisis y diseño.

Los módulos principales del sistema — autenticación, gestión de trabajos, estados, requisitos, comentarios e historial — funcionan correctamente y aplican las reglas de seguridad definidas mediante JWT y control de roles. La aplicación Flutter permite autenticación, navegación entre pantallas y gestión básica de sesión de forma estable. Las pruebas demuestran específicamente que:

- Los endpoints protegidos bloquean accesos sin autenticación o con token inválido.
- Los permisos por rol (ADMIN y DISENADOR) se aplican correctamente en todos los módulos.
- El control de pertenencia al trabajo funciona: un diseñador no puede acceder ni operar sobre trabajos en los que no participa.
- El sistema mantiene trazabilidad completa mediante el historial de estados.
- El proyecto puede ejecutarse de forma portable mediante Docker y Flutter en cualquier equipo compatible.

Veredicto: La versión actual del sistema es funcional y apta para su entrega como proyecto final del ciclo DAM 2025/2026.

17. BIBLIOGRAFÍA Y WEBGRAFÍA

17.1. ESTÁNDARES Y METODOLOGÍAS DE PRUEBAS

- IEEE Std 829-2008. IEEE Standard for Software and System Test Documentation. Institute of Electrical and Electronics Engineers.
- ISTQB Foundation Level Syllabus. (2023). International Software Testing Qualifications Board. Recuperado de https://www.istqb.org
- OWASP Testing Guide. (2021). OWASP Web Security Testing Guide v4.2. Recuperado de https://owasp.org/www-project-web-security-testing-guide/

17.2. DOCUMENTACIÓN TÉCNICA DE LAS HERRAMIENTAS

- Spring Framework. (2024). Spring Boot Testing Documentation. Recuperado de https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing
- Flutter. (2024). Testing Flutter Apps. Recuperado de https://docs.flutter.dev/testing
- Postman Inc. (2024). Postman API Platform Documentation. Recuperado de https://learning.postman.com/docs/
- Swagger. (2024). OpenAPI Specification 3.0. Recuperado de https://swagger.io/specification/
- JUnit 5. (2024). JUnit 5 User Guide. Recuperado de https://junit.org/junit5/docs/current/user-guide/
- Mockito. (2024). Mockito Framework. Recuperado de https://site.mockito.org

17.3. RECURSOS COMPLEMENTARIOS

- Baeldung. (2024). Testing in Spring Boot. Recuperado de https://www.baeldung.com/spring-boot-testing
- Baeldung. (2024). Spring Security Test. Recuperado de https://www.baeldung.com/spring-security-integration-tests
- Flutter Community. (2024). Widget Testing in Flutter. Recuperado de https://docs.flutter.dev/testing/overview
- MariaDB. (2024). MariaDB Test Documentation. Recuperado de https://mariadb.com/kb/en/testing/